

TP Guidé — Jour 2

Analyse exploratoire et représentation du texte en Python

Module NLP — Natural Language Processing — Bloc 2 · Durée : 3h30

Enseignant : Dr. Abdelhak TOUITI

Objectifs du TP

- Reprendre le corpus nettoyé au Jour 1 et calculer des indicateurs d'analyse exploratoire (EDA).
- Identifier les mots dominants et la répartition des classes (notes).
- Vectoriser le corpus avec TF-IDF (`scikit-learn`).
- Construire des embeddings de phrases avec `Sentence-Transformers` et calculer une similarité sémantique.
- Documenter vos choix — ce document devient la **Section 2 de votre dossier technique NLP**.

1. Préparation de l'environnement (PowerShell)

Reprenez le dossier TP_Jour1 créé hier, ou créez TP_Jour2 à côté. Dans un terminal PowerShell :

```
# Se déplacer dans le dossier de travail (adaptez le chemin si besoin)
cd TP_Jour1

# Reactiver l'environnement virtuel cree au Jour 1
.\venv\Scripts\Activate.ps1

# Installer les nouvelles librairies necessaires aujourd'hui
pip install scikit-learn matplotlib sentence-transformers
```

Remarque

`sentence-transformers` télécharge automatiquement un modèle pré-entraîné la première fois qu'on l'utilise (quelques dizaines de Mo, une seule fois). Comme hier, tous les outils utilisés aujourd'hui sont **gratuits et open source** — aucun compte ni service payant requis.

Créez un nouveau notebook TP_Jour2.ipynb dans le même dossier.

2. Reprise du corpus nettoyé

Pour repartir sur des bases identiques (même si vous rejoignez la séance sans votre travail d'hier), cette cellule reconstruit rapidement le corpus nettoyé du Jour 1.

```
1 import pandas as pd, re, spacy
2
3 nlp = spacy.load("fr_core_news_sm")
4 stop_words = nlp.Defaults.stop_words - {"ne", "pas", "non", "sans"}
5
6 def clean_text(text):
7     text = text.lower()
```

```

8     text = re.sub(r"<.*?>", " ", text)
9     text = re.sub(r"http\S+", " ", text)
10    text = re.sub(r"[!]{2,}", "!", text)
11    text = re.sub(r"[.]{2,}", ".", text)
12    text = re.sub(r"\s+", " ", text).strip()
13    return text
14
15    df = pd.read_csv("corpus_avis.csv")
16    df["avis_clean"] = df["avis"].apply(clean_text)
17    df = df.drop_duplicates(subset="avis_clean").reset_index(drop=True)
18    df["tokens_final"] = df["avis_clean"].apply(
19        lambda t: [tok.lemma_ for tok in nlp(t) if tok.is_alpha and tok.
20                   lemma_ not in stop_words]
21    )
22    print(df.shape)

```

✓ Point de contrôle

df.shape doit afficher (17, 4) : 17 avis, et les colonnes avis, note, avis_clean, tokens_final.

3. Analyse exploratoire : structure du corpus

```

1    df["longueur"] = df["avis_clean"].apply(lambda t: len(t.split()))
2
3    print("Nombre de documents :", len(df))
4    print("Longueur moyenne (mots) :", round(df["longueur"].mean(), 1))
5    print("Longueur mediane (mots) :", df["longueur"].median())
6    print("Longueur min / max :", df["longueur"].min(), "/", df["longueur"]
7          .max())

```

À faire : affichez également df["note"].value_counts(). Les classes vous semblent-elles équilibrées ? Notez votre réponse pour la documentation.

4. Analyse exploratoire : vocabulaire et fréquences

```

1    from collections import Counter
2    import matplotlib.pyplot as plt
3
4    tous_les_tokens = [tok for tokens in df["tokens_final"] for tok in
5                       tokens]
6    vocabulaire = set(tous_les_tokens)
7    frequences = Counter(tous_les_tokens)
8
9    print("Taille du vocabulaire :", len(vocabulaire))
10   print("Top 10 des mots les plus frequents :")
11   print(frequences.most_common(10))
12
13   top10 = frequences.most_common(10)
14   mots, comptes = zip(*top10)
15   plt.figure(figsize=(7, 4))
16   plt.bar(mots, comptes, color="#0B5E59")
17   plt.xticks(rotation=45, ha="right")
18   plt.title("Top 10 des mots les plus frequents")

```

```

18 plt.tight_layout()
19 plt.savefig("top_mots.png", dpi=120)
20 plt.show()

```

✓ Point de contrôle

Un fichier `top_mots.png` doit apparaître dans votre dossier de travail : un graphique à barres des 10 mots les plus fréquents du corpus.

5. Vectorisation TF-IDF

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 vectorizer = TfidfVectorizer()
4 X_tfidf = vectorizer.fit_transform(df["avis_clean"])
5
6 print("Dimensions de la matrice TF-IDF :", X_tfidf.shape)
7 print("Nombre de termes du vocabulaire TF-IDF :", len(vectorizer.
    get_feature_names_out()))

```

À faire : pour le document d'indice 0, affichez les 5 termes ayant le score TF-IDF le plus élevé (indice : convertissez la ligne en tableau avec `X_tfidf[0].toarray()[0]`, associez chaque valeur à `vectorizer.get_feature_names_out()`, puis triez).

6. Embeddings de phrases avec Sentence-Transformers

```

1 from sentence_transformers import SentenceTransformer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
5 embeddings = model.encode(df["avis_clean"].tolist())
6
7 print("Dimensions des embeddings :", embeddings.shape)

```

✓ Point de contrôle

`embeddings.shape` doit afficher (17, 384) : 17 avis, chacun représenté par un vecteur de 384 dimensions.

```

1 # TODO : choisissez deux avis que vous jugez proches en sens (par ex.
    deux avis positifs)
2 i, j = 0, 5 # <-- remplacez par les indices de votre choix
3 sim = cosine_similarity([embeddings[i]], [embeddings[j]])[0][0]
4 print(df["avis_clean"][i])
5 print(df["avis_clean"][j])
6 print("Similarite cosinus :", round(sim, 3))

```

7. TF-IDF vs embeddings : recherche sémantique

On compare les deux représentations sur une requête qui ne partage presque aucun mot exact avec le corpus.

```

1 requete = "je suis tres content du service propose"
2
3 # Encodage de la requete dans les deux espaces
4 requete_tfidf = vectorizer.transform([requete])
5 requete_embed = model.encode([requete])
6
7 # Similarite avec chaque avis du corpus
8 sim_tfidf = cosine_similarity(requete_tfidf, X_tfidf)[0]
9 sim_embed = cosine_similarity(requete_embed, embeddings)[0]
10
11 meilleur_tfidf = sim_tfidf.argmax()
12 meilleur_embed = sim_embed.argmax()
13
14 print("Meilleur resultat (TF-IDF) :", df["avis_clean"][meilleur_tfidf],
15       "| score :", round(sim_tfidf[meilleur_tfidf], 3))
16 print("Meilleur resultat (embeddings) :", df["avis_clean"][
17       meilleur_embed],
18       "| score :", round(sim_embed[meilleur_embed], 3))

```

Remarque

Si les deux méthodes renvoient un résultat différent, c'est normal et attendu : TF-IDF cherche des mots en commun, les embeddings cherchent un **sens** proche même sans recouvrement lexical exact. Notez ce que vous observez — c'est un point à discuter dans votre dossier.

8. Vérifier l'exploitabilité des représentations

À faire : vérifiez et notez les points suivants pour votre documentation :

- La dimension des vecteurs TF-IDF est-elle la même pour tous les documents ? (`X_tfidf.shape[0]` doit correspondre au nombre d'avis)
- Y a-t-il des vecteurs d'embeddings nuls ou incohérents ? (`embeddings.sum(axis=1)` ne doit contenir aucun 0)
- Quelle représentation retiendriez-vous si l'objectif métier était de *classer* les avis par note ? Et si l'objectif était de *retrouver* des avis similaires à une requête libre ?

9. Documenter votre analyse

Complétez cette cellule Markdown à la fin de votre notebook — elle constitue la Section 2 de votre dossier technique.

Indicateurs EDA : longueur moyenne, taille du vocabulaire
 Répartition des classes (notes) :
 Mots dominants observés :
 Représentation(s) retenue(s) et pourquoi :
 Observation TF-IDF vs embeddings :
 Limites identifiées à ce stade :

10. Pour aller plus loin (optionnel)

- Affichez les 5 avis les plus proches (embeddings) d'un avis négatif de votre choix.
- Testez un autre modèle Sentence-Transformers et comparez les scores de similarité obtenus.
- Réduisez les embeddings en 2 dimensions (ex. PCA de `scikit-learn`) et visualisez le nuage de points : les avis positifs et négatifs se séparent-ils visuellement ?

Livrable attendu

À la fin de la séance, déposez votre notebook `TP_Jour2.ipynb` (code exécuté + graphique `top_mots.png` + cellule de documentation complétée). Il constitue la **Section 2 — Analyse exploratoire et représentation** de votre dossier technique NLP, qui sera complété le Jour 3 (modélisation et évaluation).